

Group Assignment 2:

1. Introduction

In their seminal 2000 paper, *"An Evaluation of Naive Bayesian Anti-Spam Filtering,"* Androutsopoulos et al. introduced the LingSpam corpus to rigorously test whether a Naive Bayesian classifier could reliably filter unsolicited bulk email ("spam"). While Naive Bayes is a classic baseline for text classification, spam filtering introduces a unique challenge: asymmetric costs. Misclassifying a legitimate email (Ham) as Spam (a False Positive, FP) is generally far worse than letting a Spam email slip into the inbox (a False Negative, FN). In this assignment, you will replicate the core experiments of the 2000 paper.

The LingSpam corpus consists of 2,893 messages collected from the Linguist List, <https://linguistlist.org/>, mixed with spam emails. The corpus is traditionally split into 10 folds for cross-validation and is available in four versions:

- a. **Bare:** No lemmatization, stop-words included.
- b. **Stop:** No lemmatization, stop-words removed.
- c. **Lemm:** Lemmatized, stop-words included.
- d. **Lemm_Stop:** Lemmatized, stop-words removed.

2. Objectives

- i. Implement and evaluate a Naïve Bayes Classifier of text data.
- ii. Understand and apply text preprocessing such as lemmatization and stop-list filtering.
- iii. Analyse the effect of attribute-set size and training size on model performance.
- iv. Implement cost-sensitive metrics to reflect real world email filtering risks.

3. Tasks

Task 1: Data Preprocessing & Baseline Exploration

1. Download the LingSpam corpus from PutraBlast.
2. Write a program to parse the emails, separating the subject line and body from the labels (spam vs. legitimate).
3. Implement a feature extraction pipeline that converts text into binary or term-frequency vectors based on the N most frequent words (excluding stop-words if applicable).

Task 2: Classifier Implementation

Implement a Naive Bayes classifier from scratch by implementing LEARN_NAIVE_BAYES_TEXT and CLASSIFY_NAIVE_BAYES_TEXT taken from the book Machine Learning by Tom M. Mitchell page 183 and shown below:

LEARN_NAIVE_BAYES_TEXT(*Examples*, *V*)

1. *collect all words and other tokens that occur in Examples*
- *Vocabulary* $\leftarrow$ all distinct words and other tokens in *Examples*
2. *calculate the required $P(v_j)$ and $P(w_k|v_j)$ probability terms*
- For each target value v_j in *V* do
 - docs_j* $\leftarrow$ subset of *Examples* for which the target value is v_j
 - $P(v_j) \leftarrow \frac{|docs_j|}{|Examples|}$
 - Text_j* $\leftarrow$ a single document created by concatenating all members of *docs_j*
 - $n \leftarrow$ total number of words in *Text_j* (counting duplicate words multiple times)
 - for each word w_k in *Vocabulary*
 - * $n_k \leftarrow$ number of times word w_k occurs in *Text_j*
 - * $P(w_k|v_j) \leftarrow \frac{n_k+1}{n+|Vocabulary|}$

CLASSIFY_NAIVE_BAYES_TEXT(*Doc*)

- *positions* $\leftarrow$ all word positions in *Doc* that contain tokens found in *Vocabulary*
- Return v_{NB} , where

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_{i \in \text{positions}} P(a_i|v_j)$$

- You can access the book to read the whole section from the following link:
<https://www.cs.cmu.edu/~tom/files/MachineLearningTomMitchell.pdf>
- In the algorithm, $V = \{\text{Spam}, \text{Ham}\}$ in our spam filtering case. v_j represents the specific class label within V . For example, if $v_j = \text{Spam}$, $P(V_j) = \frac{|docs_j|}{|Examples|} = \frac{\text{no.of spam email documents}}{\text{Total no.of documents}}$.
- Ensure your model includes Laplace smoothing to handle unseen words during testing.

Task3: Measurements implementation

Following the paper, we define a cost factor λ . If $\lambda = 1$, a False Positive (FP) and a False Negative (FN) have equal weight. If $\lambda = 9$, misclassifying a legitimate email is 9 times worse than missing a spam email. Implement the following metrics:

- a. Weighted Accuracy, (WAcc): $Wacc = \frac{\lambda.TN + TP}{\lambda(TN + FP) + (TP + FN)}$
- b. Spam Precision (P) and Spam Recall (R).
- c. Total Cost Ratio: $TCR_\lambda = \frac{\text{No.of spam emails in the test set}}{FN + (\lambda.FP)}$

TCR value dictates the viability of your classifier. If $TCR > 1$, Your spam filter is beneficial. The cost of utilizing it is less than manual labor required to delete all the spam yourself. If $TCR < 1$, your spam filter is worse than useless. It blocks so many ham emails that you are better off turning the spam filter off entirely.

To account for λ , your classifier should only classify an email as spam if:

$$\frac{P(\text{Spam} | \text{Email})}{P(\text{Ham} | \text{Email})} > \lambda$$

Task 4: Experiments

Run the following four experiments using 10-fold cross-validation:

1. Effect of Attribute-Set Size: Fix the training size, and plot the Weighted Accuracy and TCR for $\lambda = 1, 9, 99$, against vocabulary sizes of $N = [50, 100, 500, 1000, 5000, \text{all}]$
2. Effect of Preprocessing: Compare the performance across the 4 versions of the LingSpam corpus (Bare, Stop, Lemm, Lemm_Stop). Does lemmatization significantly help?
3. Effect of Training-Corpus Size: Using the best-performing preprocessing configuration, track how Weighted Accuracy and TCR scale as the training set grows from 10% to 100%.

4. Roles and Responsibilities.

Claim your roles for this project by having a group discussion and fill out the google project sign-up sheet declaring your role:

https://docs.google.com/spreadsheets/d/1BvOoptfMriLbPQBPM_RYIX26QS-FtxdUFiJwM3Oi1DU/edit?usp=sharing

5. Deliverables

Submit a report (PDF) containing the following:

1. Abstract and Introduction. This should be a summary of your implementation and your high-level findings.
2. Experiment results:
 - a. Graph of Weighted Accuracy for $\lambda = 1, 9, 99$ vs vocabulary size.
 - b. Graph of TCR for $\lambda = 1, 9, 99$ vs vocabulary size.
 - c. Table of comparative performance (Precision, Recall, $Wacc_1, Wacc_9, Wacc_{99}, TCR_9, TCR_{99}$) across the four versions (Bare, Stop, Lemm, Lemm_Stop).
 - d. Graph of the learning curve (Training size vs Performance): TCR for the various λ against training size.
 - e. Explain the difference between your Weighted Accuracy graphs and your TCR graphs. Did any configuration achieve high accuracy while yielding poor TCR? What does this tell you about using accuracy alone to evaluate a spam filter?
3. Provide a clean well documented code or a link to a reproducible repository.
4. Only the leader of each group should submit the PDF and the link in PutraBlast.